

Name of the Student	N.AKSHAYA
Reg. No	192425129
GitHub Link	https://github.com/akshaya24032007/MLA0405-DeepLearning/tree/main/Assessment/CO5

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning
SLOT : D

COURSE CODE: MLA0405
Faculty : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35%
Assessment Tool Weightage: 30%

Duration: 90 minutes
Mark : 25

Code of Conduct:

I **N.AKSHAYA(192425129)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:N.AKSHAYA

Name of the student: N.AKSHAYA

Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	Problem-Solving and Analytical Thinking (2)	Communication Skills and Clarity of Explanation (2)	Confidence, Presentation and Professional Behaviour (1)	Total (10)
Weightage	20 %	30%	20%	20 %	10 %	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Faculty Signature :

Total Marks:

QUESTIONS: 5

Total Marks:25

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

Answer:

Problem: Detect defective products from high-dimensional sensor data.

Architecture:

Input Sensor Data → Encoder → Latent Representation → Decoder → Reconstructed Data

- **Input:** High-dimensional sensor features.
- **Encoder:** Dense layers, e.g., $128 \rightarrow 64 \rightarrow 16$ neurons, ReLU activation.
- **Latent Layer:** 16-dimensional compressed representation.
- **Decoder:** $16 \rightarrow 64 \rightarrow 128 \rightarrow$ Original dimensions.
- **Activation:** ReLU in hidden layers; linear output for continuous sensor data.
- **Loss:** Mean Squared Error (MSE) between input and reconstructed data.
- **Working:** The encoder reduces dimensionality and learns important features. The decoder reconstructs the original sensor data.
- **Defect Detection:** High reconstruction error indicates a possible defective product.
- **Evaluation:** Reconstruction loss, precision, recall, and F1-score.

Expected Outcome: Compact and useful features with accurate defect detection.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

Answer:

Problem: Handle noisy patient-record features and produce robust representations.

Architecture:

Noisy Input → Encoder → Latent Representation → Decoder → Clean Reconstruction

- **Input:** Noisy healthcare features.
- **Encoder:** Dense layers such as $128 \rightarrow 64 \rightarrow 16$, ReLU.
- **Latent Layer:** 16-dimensional representation.
- **Decoder:** $16 \rightarrow 64 \rightarrow 128 \rightarrow$ Original dimensions.
- **Regularization:** Dropout and L2 weight regularization.
- **Training Objective:** Reconstruction Loss + Regularization Penalty.
- **Loss:**
Total Loss = MSE Reconstruction Loss + λ L2 Penalty
- **Working:** Regularization prevents the network from memorizing noisy training data.
- **Application:** The learned latent features can be used for downstream classification.
- **Evaluation:** Reconstruction error and classification accuracy/F1-score.

Expected Outcome: Reduced overfitting and more robust feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

Answer:

Model: Deep Belief Network (DBN) for recommendation/content generation.

Architecture:

Visible Layer → Hidden Layer 1 → Hidden Layer 2 → Hidden Layer 3

- **Visible Layer:** User/item or content features.
- **Hidden Layers:** Multiple layers learn increasingly complex representations.
- **Activation:** Sigmoid for RBM-based hidden units.
- **Learning:** Each RBM is pretrained layer-by-layer using Contrastive Divergence.
- **Fine-tuning:** The complete network is further trained for the target application.
- **Generation:** Hidden representations are used to reconstruct or generate suitable content.

- **Justification:** DBN can learn hierarchical features from complex, high-dimensional data.
- **Evaluation:** Reconstruction error, recommendation accuracy, or generation quality.

Expected Outcome: Effective learning of complex data distributions and useful generated representations.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

Answer:

Model: Contractive Autoencoder for image/signal processing.

Architecture:

Input Image/Signal → Encoder → Latent Representation → Decoder → Reconstruction

- **Input:** Image or signal data.
- **Encoder:** Dense/convolutional layers with ReLU.
- **Latent Layer:** Compact feature representation.
- **Decoder:** Reconstructs the original input.
- **Objective:**

$$\text{Loss} = \text{Reconstruction Loss} + \lambda \|\text{Jacobian of Encoder}\|^2$$
- **Contractive Mechanism:** Penalizes large changes in the latent representation caused by small input variations.
- **Robustness:** Produces stable features even when the input contains small noise or variations.
- **Evaluation:** Reconstruction error and robustness under noisy/perturbed inputs.

Expected Outcome: Stable and noise-resistant representations.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Answer:

Problem: Generate realistic synthetic training images when real images are limited.

Architecture:

Random Noise → **Generator** → Synthetic Image → **Discriminator**

↑

Real Image

- **Generator:** Takes random noise and generates synthetic images.
- **Discriminator:** Classifies images as real or generated.
- **Training:** Generator tries to fool the discriminator, while discriminator learns to distinguish real and fake images.
- **Generator Loss:** Encourages realistic generated samples.
- **Discriminator Loss:** Measures correct real/fake classification.
- **Adversarial Process:** Both networks are trained iteratively until generated images become realistic.
- **Evaluation:** Image quality, discriminator performance, and similarity/diversity of generated samples.
- **GAN vs Autoencoder:** Autoencoders mainly focus on reconstruction, whereas GANs directly optimize realistic sample generation.
- **Justification:** GAN is suitable because the organization needs **new realistic synthetic images**.

Expected Outcome: Increased training data and improved model performance with realistic synthetic samples.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.

Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.
---------	----	---	---	----------------------------------	---